



TARGET-BUSINESS ANALYSIS

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset
 1. Data type of columns in a table
 2. Time period for which the data is given
 3. Cities and States of customers ordered during the given period

1. DATATYPE OF COLUMNS OF THE SELLER TABLE

The screenshot shows the Google Cloud BigQuery console. The 'seller' table schema is displayed in the center. The columns and their data types are as follows:

Field name	Type	Mode	Collation
seller_id	STRING	NULLABLE	
seller_zip_code_prefix	INTEGER	NULLABLE	
seller_city	STRING	NULLABLE	
seller_state	STRING	NULLABLE	

On the right, a query is executed: `SELECT * FROM `targetproject.Target_report.order_items` LIMIT 3`. The results show three rows of order items.

This image shows the data type of “seller” table. The datatype of ‘seller_id’, ‘seller_city’, ‘seller_state’ is “NULLABLE” string and the datatype of ‘seller_zip_code_prefix’ is “NULLABLE” integer.

2. Time period of given data

The screenshot shows the Google Cloud BigQuery console. The 'order_items' table schema is displayed in the center. The columns and their data types are as follows:

Field name	Type	Mode	Collation
order_id	STRING	NULLABLE	
order_item_id	INTEGER	NULLABLE	
product_id	STRING	NULLABLE	
seller_id	STRING	NULLABLE	
shipping_start_date	TIMESTAMP	NULLABLE	
price	FLOAT	NULLABLE	
discount_value	FLOAT	NULLABLE	

On the right, a query is executed: `SELECT seller_id, shipping_start_date FROM `targetproject.Target_report.order_items` LIMIT 3`. The results show three rows of order items with their shipping start dates.

```
SELECT max(shipping_limit_date),min(shipping_limit_date) FROM `target1project.Target_report.order_items`
```

Row	f0_	f1_	
1	2020-04-09 22:35:08 UTC	2016-09-19 00:15:34 UTC	

By running this query we can see that this dataset is given for 2016-2020.

3. City and State of Customers Ordered during given time

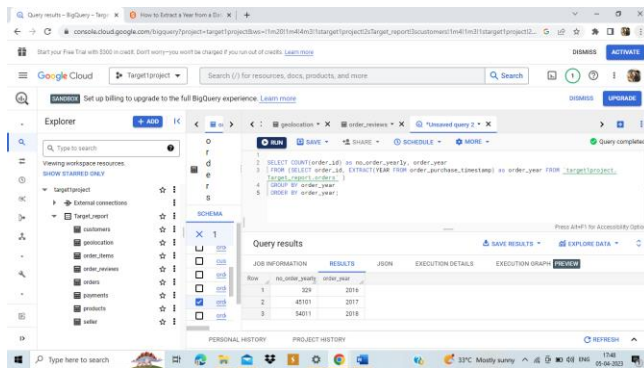
The screenshot shows the Google Cloud BigQuery console. On the left, the 'Explorer' pane shows the 'customers' table under the 'Target_report' dataset. The main pane displays the 'customers' table schema with columns: customer_id (STRING, NULLABLE), customer_unique_id (STRING, NULLABLE), customer_zip_code_prefix (INTEGER, NULLABLE), customer_city (STRING, NULLABLE), and customer_state (STRING, NULLABLE). The 'Query results' pane on the right shows the results of a query: 'SELECT customer_city, customer_state FROM `target1project.Target\_report.customers` GROUP BY customer_city, customer_state ORDER BY customer_state'. The results show 50 rows of customer data, including cities like 'porto calvo' and 'boca da mata' from the state of 'AL'.

```
SELECT customer_city, customer_state FROM `target1project.Target_report.customers`
GROUP BY customer_city, customer_state
ORDER BY customer_state;
```

From the above query, we can see that there are customers from 2 different states who lives in 50 different cities.
And there are 99441 customers.

1. In-depth Exploration:

1. Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?
2. What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night)?



```
SELECT COUNT(order_id) as no_order_yearly, order_year
FROM (SELECT order_id, EXTRACT(YEAR FROM order_purchase_timestamp) as order_year
FROM `target1project.Target_report.orders` )
GROUP BY order_year
ORDER BY order_year;
```

This query shows that no. of orders placed every year is increasing

Row	no_order_yearly	order_year
1	329	2016
2	45101	2017
3	54011	2018

From 2016 to 2017 , there's a hike of 127% and sales grew exponentially, From 2017 to 2018 sales growth was 16 % though it is quite less but it is in positive direction so we can say that Target was able to hold its place in the market. The reason for decrease in growth rate in 2018 can be increase in competition.

Also, from the query below

```
SELECT COUNT(order_id) as no_order_monthly, order_month
FROM (SELECT order_id, EXTRACT(MONTH FROM order_purchase_timestamp) as order_month
FROM `target1project.Target_report.orders` )
GROUP BY order_month
ORDER BY no_order_monthly;
```

And the output shows the sales was highest in the month of August, May and July and it was lowest in the month of September which is quite surprising as the expected month was December. It maybe possible there was "seasonal -sale, with heavy discounts" in the month of August which led to peak in the month of August.

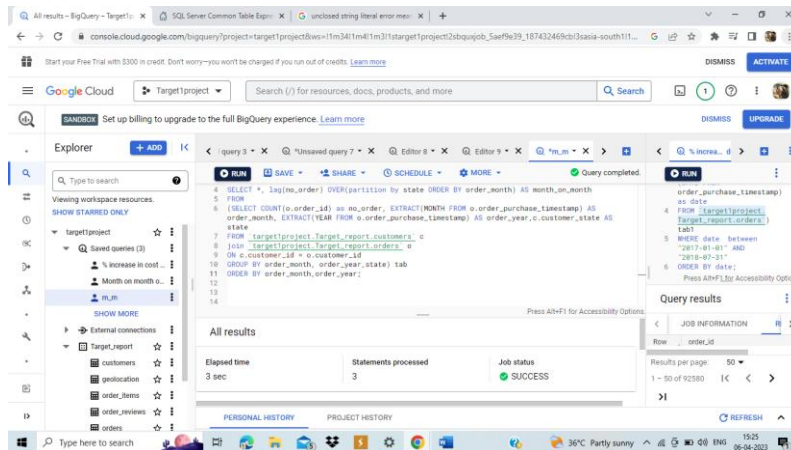
2. The below Query

```
SELECT COUNT(order_id) as no_order_day, order_time
FROM (SELECT order_id, EXTRACT(time FROM order_purchase_timestamp) as order_time
FROM `target1project.Target_report.orders` )
GROUP BY order_time
ORDER BY no_order_day desc;
```

Shows Brazilian customer tend to buy more in the afternoon "15:59:34" as maximum customers buy in this hour of the day.

3. Evolution of E-commerce orders in the Brazil region:
 1. Get month on month orders by states
 2. Distribution of customers across the states in Brazil

Month on Month orders by states



```
SELECT *, lag(no_order) OVER(partition by state ORDER BY order_month) AS month_on_m
onth
FROM
(SELECT COUNT(o.order_id) as no_order, EXTRACT(MONTH FROM o.order_purchase_timestam
p) AS order_month, EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,c.cu
stomer_state AS state
FROM `target1project.Target_report.customers` c
join `target1project.Target_report.orders` o
ON c.customer_id = o.customer_id
GROUP BY order_month, order_year,state) tab
ORDER BY order_month,order_year;
```

With the help of above code we can get month on month orders by state.

Row	no_order	order_month	order_year	state	month_on_month	Load more
3	25	1	2017	BA	239	
4	13	1	2017	DF	138	
5	31	1	2017	SC	314	
6	11	1	2017	MT	85	
7	54	1	2017	RS	373	
8	2	1	2017	PB	31	
9	97	1	2017	RJ	893	
10	12	1	2017	ES	147	

DISTRIBUTION OF CUSTOMERS OVER STATES OF BRAZIL

```
SELECT COUNT(customer_id) AS no_customers, customer_state
FROM `target1project.Target_report.customers`
group by customer_state
ORDER BY no_customers ;
```

From the above query we can get no. of customers placing order on Target from different state of Brazil.

The screenshot shows the Google Cloud BigQuery console interface. The left sidebar displays the Explorer with a tree view of resources including 'target1project', 'Saved queries (4)', and 'Target_report'. The main area shows a query editor with the following SQL code:

```
1 SELECT MAX(customer_state) as max, MIN(customer_state) min
2 FROM `target1project.Target_report.customers` ;
3
```

Below the query editor, the 'Query results' section is visible, showing a table with columns 'max' and 'min'. The results are as follows:

Row	max	min
1	TO	AC

The right sidebar shows the 'Query results' section with a table containing 'order_id' and a 'Results per page' dropdown set to 50. The bottom of the screen shows the Windows taskbar with the date and time as 15:54 on 06-04-2023.

```
SELECT MAX(customer_state) as max, MIN(customer_state) min
FROM `target1project.Target_report.customers` ;
```

From this query, we can see that maximum customers are from TO and lowest no. of customers are from AC state.

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.
 1. Get % increase in cost of orders from 2017 to 2018 (include months between Jan to Aug only) - You can use "payment_value" column in payments table
 2. Mean & Sum of price and freight value by customer state

1.

The screenshot shows the Google Cloud BigQuery console interface. On the left is the Explorer pane showing the project structure. The main area displays a SQL query and its results. The query calculates the percentage increase in total payment from 2017 to 2018. The results table shows two rows: one for 2017 with a total payment of 3669022.11 and a null percentage increase, and one for 2018 with a total payment of 8694733.83 and a percentage increase of 136.97687164666226.

```
SELECT *, ((total_payment - LAG(total_payment) over(order by total_payment))/ LAG(t
otal_payment) over(order by total_payment)) * 100 as per_increase
from
```

```
(SELECT order_year, SUM(pay_val) total_payment
FROM
(SELECT o.order_id, EXTRACT (year FROM o.order_purchase_timestamp) order_year, EXTR
ACT(MONTH FROM o.order_purchase_timestamp) order_month, p.payment_value as pay_val
FROM `target1project.Target_report.orders` o
JOIN `target1project.Target_report.payments` p
ON o.order_id = p.order_id
)
WHERE order_year between 2017 and 2018 AND order_month between 1 and 8
GROUP BY order_year )
order by order_year
```

From the above query we can find percentage increase in cost in 2018 from 2017 between January to august.

Row	order_year	total_payment	per_increase
1	2017	3669022.1199999228	null
2	2018	8694733.83999998639	136.97687164666226

It is 136.97%.

2.

The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'Explorer' pane displays a project named 'Target1project' with various datasets like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'seller'. The main area shows a SQL query being executed. The query is a SELECT statement that calculates the sum and average of price and freight value, grouped by state. The results are displayed in a table with columns: Row, sum_price, mean_price, sum_freight_val, mean_freight_val, and state. The results show data for two rows, both from the state of MA.

Row	sum_price	mean_price	sum_freight_val	mean_freight_val	state
1	156453.529...	148.297184...	29715.4300...	28.1662843...	MT
2	119648.219...	145.204150...	31523.7700...	38.2570024...	MA

```
SELECT SUM(oi.price) sum_price, AVG(oi.price) mean_price , SUM(freight_value) sum_f  
reight_val, AVG(freight_value) mean_freight_val , c.customer_state as state
```

```
FROM `target1project.Target_report.order_items` oi  
JOIN `target1project.Target_report.orders` o  
ON oi.order_id = o.order_id  
JOIN `target1project.Target_report.customers` c  
ON o.customer_id = c.customer_id  
GROUP BY state;
```

From the above query we can get sum and mean of price and freight value.

5. Analysis on sales, freight and delivery time

1. Calculate days between purchasing, delivering and estimated delivery
 2. Find time_to_delivery & diff_estimated_delivery. Formula for the same given below:
 - $\text{time_to_delivery} = \text{order_purchase_timestamp} - \text{order_delivered_customer_date}$
 - $\text{diff_estimated_delivery} = \text{order_estimated_delivery_date} - \text{order_delivered_customer_date}$
 3. Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery
 4. Sort the data to get the following:
 5. Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5
 6. Top 5 states with highest/lowest average time to delivery
 7. Top 5 states where delivery is really fast/ not so fast compared to estimated date.
-

The screenshot shows the Google Cloud BigQuery console. The query editor contains the following SQL code:

```
SELECT date_diff(d, o, day) AS actual_date, date_diff(e, o, day) AS expected_date
FROM
(
  SELECT EXTRACT(DATE FROM order_purchase_timestamp) as o, EXTRACT(DATE FROM order_delivered_customer_date) as d,
  EXTRACT(DATE FROM order_estimated_delivery_date) as e
  FROM `target1project.Target_report.orders`
)
```

The query results are displayed in a table with the following data:

Row	actual_date	expected_date
1	null	51
2	null	7
3	null	45

The console also shows the Explorer panel on the left with a list of workspace resources including customers, geolocation, order_items, order_reviews, orders, payments, products, and seller.

5.1.

```
SELECT date_diff(d, o, day) AS actual_date, date_diff(e, o, day) AS expected_date
FROM
(SELECT EXTRACT(DATE FROM order_purchase_timestamp) as o, EXTRACT(DATE FROM order_d
elivered_customer_date) as d,
EXTRACT(DATE FROM order_estimated_delivery_date) as e

FROM `target1project.Target_report.orders`)
```

From the above query we can calculate days between purchasing, delivery and estimated delivery.

5.2.

The screenshot shows the Google Cloud BigQuery console. The query editor contains the following SQL code:

```
SELECT datetime_diff(order_purchase_timestamp,order_delivered_customer_date, hour) as time_to_delivery,
datetime_diff(order_estimated_delivery_date,order_delivered_customer_date,hour) as diff_estimated_delivery
FROM `target1project.Target_report.orders`;
```

The query results are displayed in a table with the following data:

Row	time_to_delivery	diff_estimated_delivery
1	-168	1088
2	-722	-310
3	-743	681

The console also shows the Explorer panel on the left with a list of workspace resources including customers, geolocation, order_items, order_reviews, orders, payments, products, and seller.

```
SELECT datetime_diff(order_purchase_timestamp,order_delivered_customer_date, hour)
as time_to_delivery,
datetime_diff(order_estimated_delivery_date,order_delivered_customer_date,hour) as
diff_estimated_delivery FROM `target1project.Target_report.orders`;
```

5.3

The screenshot shows the Google Cloud BigQuery console interface. The top navigation bar includes the Google Cloud logo, a search bar, and a 'SANDBOX' button. The left sidebar contains an 'Explorer' panel with a search bar and a list of workspace resources, including 'External connections', 'Target_report', 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'seller'. The main panel displays a SQL query in the editor, which has been executed. The query results are shown in a table format with columns: 'Row', 'avg_freight_val', 'avg_time_to_del', and 'avg_diff_estimaj'. The results table contains three rows of data. The bottom status bar shows the system time as 23:01 on 06-04-2023.

```

1 SELECT Avg(oi.freight_value) as avg_freight_val, avg(datetime_diff(o.order_purchase_timestamp,o.order_delivered_customer_date, hour))
   AS avg_time_to_delivery, avg(datetime_diff(order_estimated_delivery_date,order_delivered_customer_date, hour)) as
   avg_diff_estimated_delivery
2 FROM `target1project.Target_report.orders` as o
3 join `target1project.Target_report.order_items` as oi
4 ON o.order_id = oi.order_id
5 join `target1project.Target_report.customers` as c
6 ON o.customer_id = c.customer_id
7 GROUP BY customer_state;

```

Row	avg_freight_val	avg_time_to_del	avg_diff_estimaj
1	28.1662843...	-430.55930...	333.063645...
2	38.2570024...	-519.05875...	221.117500...
3	35.8436711...	-587.22716...	193.133489...

```

SELECT Avg(oi.freight_value) as avg_freight_val, avg(datetime_diff(o.order_purchase
_timestamp,o.order_delivered_customer_date, hour)) AS avg_time_to_delivery, avg(date
time_diff(order_estimated_delivery_date,order_delivered_customer_date, hour)) as avg
_diff_estimated_delivery
FROM `target1project.Target_report.orders` as o
join `target1project.Target_report.order_items` as oi
ON o.order_id = oi.order_id
join `target1project.Target_report.customers` as c
ON o.customer_id = c.customer_id
GROUP BY customer_state;

```

5.4

```

SELECT row_number() OVER (ORDER BY avg_freight_val) AS top_5_state, avg_freight_val,
state
from
(SELECT AVG(oi.freight_value) as avg_freight_val, c.customer_state as state
FROM `target1project.Target_report.orders` as o
join `target1project.Target_report.order_items` as oi
ON o.order_id = oi.order_id
join `target1project.Target_report.customers` as c
ON o.customer_id = c.customer_id
GROUP BY state) tab1
ORDER BY top_5_state ASC

LIMIT 5;

```

Query results - BigQuery - Target1project

Start your Free Trial with \$300 in credit. Don't worry—you won't be charged if you run out of credits. [Learn more](#)

Google Cloud Target1project Search (/) for resources, docs, products, and more

SANDBOX Set up billing to upgrade to the full BigQuery experience. [Learn more](#)

Explorer

Viewing workspace resources. SHOW STARRED ONLY

External connections

Target_report

customers

geolocation

order_items

order_reviews

orders

payments

products

seller

Query results

Query completed.

```

1 SELECT row_number() OVER (ORDER BY avg_freight_val) AS top_5_state, avg_freight_val, state
2 FROM
3 (SELECT AVG(o1.freight_value) AS avg_freight_val, c.customer_state AS state
4 FROM `target1project.Target_report.orders` AS o
5 JOIN `target1project.Target_report.order_items` AS oi
6 ON o.order_id = oi.order_id
7 JOIN `target1project.Target_report.customers` AS c
8 ON o.customer_id = c.customer_id
9 GROUP BY state) tab1
10 ORDER BY top_5_state ASC
11

```

Query results

JOB INFORMATION RESULTS JSON EXECUTION DETAILS EXECUTION GRAPH PREVIEW

Row	top_5_state	avg_freight_val	state
1	1	15.1472753...	SP
2	2	20.5316515...	PR
3	3	20.6301668...	MG

PERSONAL HISTORY PROJECT HISTORY REFRESH

5.5

Query results - BigQuery - Target1project

Start your Free Trial with \$300 in credit. Don't worry—you won't be charged if you run out of credits. [Learn more](#)

Google Cloud Target1project Search (/) for resources, docs, products, and more

SANDBOX Set up billing to upgrade to the full BigQuery experience. [Learn more](#)

Explorer

Viewing workspace resources. SHOW STARRED ONLY

External connections

Target_report

customers

geolocation

order_items

order_reviews

orders

payments

products

seller

Query results

Query completed.

```

2 FROM
3 (SELECT avg(datetime_diff(o.order_purchase_timestamp, o.order_delivered_customer_date, hour)) AS avg_time_to_delivery ,
4 c.customer_state AS state
5 FROM `target1project.Target_report.orders` AS o
6 JOIN `target1project.Target_report.order_items` AS oi
7 ON o.order_id = oi.order_id
8 JOIN `target1project.Target_report.customers` AS c
9 ON o.customer_id = c.customer_id
10 GROUP BY state) tab1
11 ORDER BY top_5_state ASC
12
13 LIMIT 5;
14

```

Query results

JOB INFORMATION RESULTS JSON EXECUTION DETAILS EXECUTION GRAPH PREVIEW

Row	top_5_state	avg_time_to_del	state
2	2	-676.45679...	AP
3	3	-632.84662...	AM

PERSONAL HISTORY PROJECT HISTORY REFRESH

```

SELECT row_number() OVER (ORDER BY avg_time_to_delivery) AS top_5_state, avg_time_to_delivery, state
from
(SELECT avg(datetime_diff(o.order_purchase_timestamp,o.order_delivered_customer_date, hour)) AS avg_time_to_delivery ,
c.customer_state as state
FROM `target1project.Target_report.orders` as o
join `target1project.Target_report.order_items` as oi
ON o.order_id = oi.order_id
join `target1project.Target_report.customers` as c
ON o.customer_id = c.customer_id
GROUP BY state) tab1
ORDER BY top_5_state ASC

```

LIMIT 5;

5.6

```

SELECT row_number() OVER (ORDER BY tab1.avg_diff_estimated_delivery) AS top_5_state, tab1.avg_diff_estimated_delivery, state
from
(SELECT avg(datetime_diff(order_estimated_delivery_date, order_delivered_customer_date, hour)) as avg_diff_estimated_delivery ,
c.customer_state as state
FROM `target1project.Target_report.orders` as o
join `target1project.Target_report.order_items` as oi
ON o.order_id = oi.order_id
join `target1project.Target_report.customers` as c
ON o.customer_id = c.customer_id
GROUP BY state) tab1
ORDER BY top_5_state ASC

```

LIMIT 5;

The screenshot shows the Google Cloud BigQuery console interface. The top navigation bar includes the Google Cloud logo, a search bar, and a notification for a free trial. The left sidebar contains an 'Explorer' panel with a search bar and a list of workspace resources, including 'Target_report' and its sub-resources like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'seller'. The main area displays a SQL query in the editor, which is a window titled '*Unsaved query 7'. The query is identical to the one provided in the text blocks. Below the editor, the 'Query results' section is visible, showing a table with columns 'top_5_state', 'avg_diff_estimated_delivery', and 'state'. The table contains two rows of data. The bottom of the screen shows a Windows taskbar with various application icons and system information like temperature and time.

Query results

Row	top_5_state	avg_diff_estimated_delivery	state
1	1	193.133489...	AL
2	2	221.117500...	MA

6. Payment type analysis:

1. Month over Month count of orders for different payment types
2. Count of orders based on the no. of payment installments.

6.2.

The screenshot shows the Google Cloud BigQuery console interface. On the left is the Explorer pane with a tree view of datasets including 'Target_report'. The main area displays a SQL query and its results. The query is:

```
1 SELECT order_id, COUNT(order_id) over(partition by payment_installments) as no_order_installments
2 FROM `target1project.Target_report.payments` ;
```

The query results are shown in a table with two columns: 'order_id' and 'no_order_installments'. The results are as follows:

Row	order_id	no_order_installments
1	b502/abe03e/bc0c18b52b43...	17
2	3e721650e845549cd3268a57...	17
3	91c4fb2a013280c780ea0810...	17
4	77ad790bc77b40f61485833a4...	17
5	9cbbfeb4725fc1144efeb735c4...	17
6	2851b4d871e918b7328baeb6...	17
7	ff2ac13b0f36dfb6af9699f59b...	17

```
SELECT order_id, COUNT(order_id) over(partition by payment_installments) as no_order_installments
FROM `target1project.Target_report.payments` ;
```

6.1.

```
SELECT COUNT(orders) no_order, p_type, month_order
FROM
  (SELECT o.order_id as orders, EXTRACT(MONTH FROM o.order_purchase_timestamp) as month_order, p.payment_type as p_type
  FROM `target1project.Target_report.payments` p
  JOIN `target1project.Target_report.orders` o
  ON p.order_id = o.order_id
  )
GROUP BY p_type, month_order
ORDER BY month_order;
```

The above query gives month over month count of orders for different payment types.

The screenshot shows the Google Cloud BigQuery console interface. The Explorer on the left lists workspace resources, including 'Target_report' with sub-items like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'seller'. The main editor displays a SQL query:

```

1 SELECT COUNT(orders) no_order, p_type, month_order
2 FROM
3 (SELECT o.order_id as orders, EXTRACT(MONTH FROM o.order_purchase_timestamp) as month_order, p.payment_type as p_type
4 FROM `target1project.Target_report.payments` p
5 JOIN `target1project.Target_report.orders` o
6 ON p.order_id = o.order_id
7 )
8 GROUP BY p_type, month_order
9 ORDER BY month_order;

```

The query results table shows the following data:

Row	no_order	p_type	month_order
9	591	voucher	3
10	7707	credit_card	3

The interface also includes a top navigation bar with 'Google Cloud' and 'Target1project', a search bar, and a bottom status bar showing the time as 00:38 on 07-04-2023.

7. Actionable insights

No. of orders with review score

The screenshot shows the Google Cloud BigQuery console interface. The Explorer on the left lists workspace resources, including 'Target_report' with sub-items like 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'seller'. The main editor displays a SQL query:

```

1 SELECT COUNT(order_id) no_orders, review_score
2 FROM `target1project.Target_report.order_reviews`
3 GROUP BY review_score
4 ORDER BY no_orders desc;

```

The query results table shows the following data:

Row	no_orders	review_score
1	57328	5
2	19142	4
3	11424	1
4	8179	3
5	3151	2

The interface also includes a top navigation bar with 'Google Cloud' and 'Target1project', a search bar, and a bottom status bar showing the time as 00:47 on 07-04-2023.

```

SELECT COUNT(order_id) no_orders, review_score
FROM `target1project.Target_report.order_reviews`
GROUP BY review_score
ORDER BY no_orders desc;

```

Row	no_orders	review_score
1	57328	5
2	19142	4
3	11424	1
4	8179	3
5	3151	2

Highest no. of orders were rated 5 which shows customers are happy with their orders. There are 11424 orders , reviewed 1 which should be taken into consideration not to just increase the sales of the company but also to maintain reputation of the company in the market.

8. RECOMMENDATION

Target is doing good at sales but there's always a room for improvement. Most of the customers are happy with their orders but significant no. of customers also reviewed "1".

Company should look upon this review precisely and try to make it up to the mark.

The screenshot shows the Google Cloud BigQuery console interface. On the left, the 'Explorer' pane lists workspace resources including 'customers', 'geolocation', 'order_items', 'order_reviews', 'orders', 'payments', 'products', and 'seller'. The main editor area displays a SQL query: `SELECT COUNT(order_id), payment_type FROM `target1project.Target_report.payments` group by payment_type`. Below the query, the 'Query results' section shows a table with 5 rows of data:

Row	fo_	payment_type
1	76795	credit_card
2	5775	voucher
3	3	not_defined
4	1529	debit_card
5	19784	UPI

The bottom of the console shows the 'PERSONAL HISTORY' and 'PROJECT HISTORY' tabs, along with a system tray at the very bottom indicating the date and time as 01:17 on 07-04-2023.

Type of payment used by different customers.

```
SELECT COUNT(order_id), payment_type FROM `target1project.Target_report.payments`
group by payment_type
```

Row	f0_	payment_type	
1	76795	credit_card	
2	5775	voucher	
3	3	not_defined	
4	1529	debit_card	
5	19784	UPI	

From this we can see most of the people use credit card as a mode of payment. Target can increase their sales by giving more offers on various card.

